Request and controller tests do not—repeat, *do not*—follow the redirect. Any data-validation tests you write apply only to the method before the redirect occurs. If you need your test to follow the redirection for some reason, you're cordially invited to try a system test; see Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151.

Older Rails Controller Tests

In Rails 5, two features of controller testing—one commonly used, one less so—were deprecated. This deprecation has the effect of limiting the scope of controller testing. In the eyes of the Rails core, this scope should be picked up by integration testing, though I'd suggest that some of it should be picked up by moving code out of the controller and unit-testing it. The following is a quick guide to what Rails 4 controller tests looked like, as you'll probably see a bunch of them.

Controller Test Requests

Rails 4 provided the same set of methods we've covered for creating a request—`get`, `post` and their friends—but the method parameters were different:

```
get :show, {id: @task.id}, {user_id: "3",
    current_project: @project.id.to_s}, {notice: "flash test"}
```

Here, the method name, `get`, is the HTTP verb being simulated—sort of. While the controller test will set the HTTP verb if for some reason you query the Rails request object, it does not check the Rails routing table to see if that action is reachable using that HTTP verb. As a result, you can't test routing via a controller test. (Refer back to *Testing Routes*, on page 218, to see how Rails provides a mechanism for testing routes.)

The first argument, in this case `:show`, is the controller action being called. The second argument, `{id: @task.id}`, is a hash that becomes the `params` hash in the controller action. In the controller action called from this test, you would expect `params[:id]` to equal `@task.id`. The Rails form name-parsing trick is not used here—if you want to simulate a form upload, you use a multilevel hash directly, as in `user: {name: "Noel", email: "noel@noelrappin.com"}`, which implies `params[:user][:name] == "Noel"` in the controller.

Any value passed in this hash argument is converted to a string—specifically, `to_param` is called on it. So you can do something like `id: 3`, confident that it will